Progetto su base mobile CoderBot
Sistemi Embedded
Fabio Lo Presti - 886388

Indice:

1 Descrizione del progetto **2**

2 Controllo dei motori **5**

2.1 Misurazione delle costanti di conversione 6

2.2 Controllo della velocità 8

3 Odometria **9**

3.1 Modifiche 9

3.2 Funzionamento 10

4 Controllo cartesiano **11**

4.1 Modifiche 12

4.2 Funzionamento 12

5 Scheduling **13**

5.1 Divisione in Task 14

5.2 Impostazione di periodi e deadline 15

5.3 Impostazione dei WCET 15

6 Fine-tuning delle costanti **17**

6.1 Velocità 17

6.2 Odometria 19

6.3 Errore reale 21

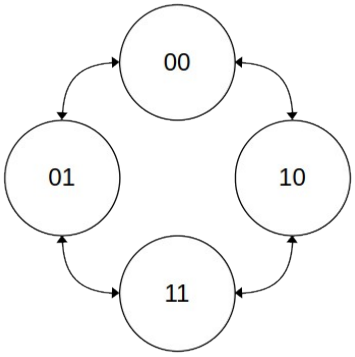
1 Descrizione del progetto

Descrizione del hardware
Il progetto è stato realizzato utilizzando la base mobile CoderBot, un piccolo robot a due ruote. A livello hardware, il robot è composto da un Raspberry Pi, una scheda HAT (Hardware Attached on Top), due motori, due encoder incrementali (IE) e un powerbank.

La scheda HAT ha il compito di alimentare i motori, poiché i pin GPIO (General Purpose Input/Output) del Raspberry Pi non forniscono tensioni sufficienti per farli funzionare direttamente. Sia la scheda HAT che il Raspberry Pi sono alimentati dal powerbank.

Il robot dispone quindi di due attuatori e due sensori. Gli attuatori sono i motori che azionano le due ruote, consentendo il movimento del robot. I sensori sono costituiti dagli encoder incrementali, collegati rispettivamente a ciascun motore.

Durante la rotazione del motore, ciascun encoder genera due segnali a onda quadra sfasati di 90 gradi tra loro. Questi segnali permettono di rilevare la direzione di rotazione del motore e di calcolare l'entità della rotazione. Le variazioni di stato dei due segnali seguono una specifica macchina a stati, illustrata nella figura seguente:



Fonte: https://jack23247.github.io/blog/embedded/quadrature/

Ogni "tick" rappresenta una transizione di stato. Il numero di tick può essere convertito in unità di misura standard, come l'angolo di rotazione in radianti o la distanza percorsa dalla ruota in millimetri, attraverso opportuni fattori di conversione.

Per ciascuna ruota sono utilizzati quattro pin GPIO: due in modalità output, per controllare la direzione di rotazione del motore, e due in modalità input, per leggere i segnali degli encoder. Attivando uno dei due pin di output, il motore ruota in una direzione; attivando l'altro, ruota in direzione opposta. I pin in input consentono la lettura delle onde quadre generate dagli encoder.

Obbiettivi del progetto
L'obiettivo del progetto è la realizzazione di codice in linguaggio C che consenta al CoderBot di percorrere un arco di circonferenza arbitrario, mantenendo una velocità specificata dal utente e compatibile con le capacità fisiche del robot.

Dettagli software
Nella relazione ho specificato quali parti del progetto non sono state svolte da me e quali sono state svolte in collaborazione con altri. Le parti senza autore indicato sono state realizzate interamente da me.

Non ho interagito direttamente con il GPIO del Raspberry Pi, ma ho utilizzato la libreria open source “coderbot” sviluppata da Jacopo Maltagliati, che a sua volta si basa sulla libreria C “pigpio”. L’unica modifica apportata al codice di Maltagliati è l’aggiunta di un mutex negli struct degli encoder delle ruote. Questo mi ha permesso di aggiornare in modo sicuro il numero di tick percorsi all’interno della funzione di callback della libreria e nel ciclo di controllo della velocità.

La libreria coderbot fornisce funzioni e strutture dati che semplificano il controllo dei motori e la lettura degli encoder, permettendo di non gestire i dettagli a basso livello. Il mio codice utilizza direttamente la libreria pigpio solo per inizializzarla e terminarla.

Il controllo dei motori avviene tramite questi passaggi:

1. Creazione di uno struct che rappresenta il motore, contenente i pin di controllo e un valore di direzione.
2. Passaggio dello struct a una funzione che inizializza i pin.
3. Impostazione della velocità del motore, passando lo struct e un valore di duty cycle a una funzione dedicata. Per fermare il motore si usa una funzione di reset, passando lo struct.

La funzione di inizializzazione configura i due pin come output, imposta il periodo del PWM e la durata massima del segnale alto nel ciclo PWM. Dopo l’inizializzazione, il motore viene controllato semplicemente impostando direzione e duty cycle. La libreria gestisce i pin automaticamente in base a questi valori.

La lettura degli encoder si svolge così:

1. Creazione di uno struct per l'encoder, contenente i pin dei segnali A e B e un indice per un pin GPIO non valido. A causa di una mia modifica, è necessario anche inizializzare un mutex.
2. Inizializzazione dell'encoder con una funzione dedicata.
3. Registrazione delle funzioni di callback, che verranno chiamate al verificarsi di cambiamenti di stato dei segnali A e B. È possibile utilizzare le callback di default fornite dalla libreria, passando ad una apposita funzione lo struct dell'encoder e un valore di timeout, oppure definire callback personalizzate. Il parametro timeout specifica il tempo massimo di attesa prima che venga gestita una condizione di timeout. Se timeout è impostato a 0, la condizione di timeout non viene mai attivata.

Lo struct dell'encoder contiene:

- i pin dei segnali A e B,
- l'indice dell'ultimo pin che ha cambiato stato (usato per il debouncing),
- un mutex per proteggere l'accesso al conteggio dei tick,
- lo stato attuale dei segnali A e B,
- la direzione di rotazione,
- il numero di tick effettuati,
- il numero di "bad tick" (transizioni non valide),
- un puntatore generico a disposizione dell'utente della libreria.

Il debouncing funziona così: se uno dei due segnali cambia stato più volte prima che cambi anche l'altro, tutti i cambiamenti dopo il primo vengono ignorati. La callback controlla che le transizioni seguano la macchina a stati finiti prevista. Se la transizione non è valida, si incrementa "bad_ticks". Se è valida, si aggiorna "tick" sommando 1 o -1 a seconda della direzione di rotazione.

I "bad tick" rappresentano transizioni errate, mentre i "tick" indicano la quantità di rotazione misurata, in unità non standard.

Per compilare il progetto è sufficiente eseguire "make" con il target di default nella cartella "/progetto". Per riportare la cartella al suo stato originale è sufficiente eseguire "make clean".

Fonti principali di errore e rumore

- Il CoderBot è molto sensibile al livello di carica del powerbank. I problemi iniziano a manifestarsi quando la carica scende da 4 a 3 tacche. Usarlo con 2 o 1 tacca è possibile, ma in queste condizioni, anche con duty cycle normali, si possono verificare i problemi descritti nella sezione 6.1, che normalmente si manifestano solo con duty cycle troppo elevati. Le costanti per calcolare il duty cycle, in oltre, risulteranno sbagliate.
- Oltre alle due ruote, il CoderBot ha una sfera di metallo come terzo punto di appoggio. Il movimento di questa sfera è imprevedibile, soprattutto se sporca. Su terreni con fughe, la sfera può incastrarsi e bloccare il robot.
- Le ruote possono slittare su superfici con basso attrito. Inoltre, non sono completamente stabili, presentano un leggero gioco che può causare imprecisioni nello spostamento. La presenza di sporco sulle ruote può peggiorare l'aderenza.
- La posizione viene aggiornata usando i tick degli encoder tramite calcoli in virgola mobile, effettuati in modo discreto e senza correzioni da altri sensori. Questo porta a un accumulo progressivo di errori di approssimazione, che aumenta con la lunghezza del percorso.

2 Controllo dei motori

A causa della scarsità di sensori sul robot, in questo progetto abbiamo fatto alcune ipotesi molto semplificate sull'ambiente:

- Il terreno è perfettamente piano.
- Non ci sono ostacoli sul percorso del robot.
- Le ruote hanno sempre presa perfetta e non slittano.

2.1 Misurazione delle costanti di conversione

Per controllare il robot, è necessario misurare quanto si muovono le ruote. I tick degli encoder sono proporzionali alla rotazione della ruota e, moltiplicandoli per una costante, si può stimare la distanza percorsa in millimetri.

La costante è stata calcolata con questo procedimento:

- Abbiamo trovato, tramite tentativi, duty cycle per i motori sinistro e destro che permettevano al robot di percorrere un arco quasi rettilineo di circa 10 cm.
- Il robot ha mostrato a schermo il numero di tick per ogni ruota, mentre noi abbiamo misurato la distanza percorsa con un metro.
- Per ogni misurazione e ruota, abbiamo diviso la distanza percorsa per il numero di tick, ottenendo un'approssimazione della costante.
- Infine, abbiamo calcolato la media delle costanti ottenute.

Per il CoderBot "icecube3" abbiamo effettuato due misurazioni:

- La prima in laboratorio, da me, Leonardo Bizzoni, Camilla Cantaluppi e Paolo Strianese. In questa fase la precisione era limitata al centimetro e questi valori non sono utilizzati nel progetto finale.
- La seconda in aula studio, da me e Leonardo Bizzoni, con un metro preciso al millimetro. I valori ottenuti sono i seguenti:

Lunghezza del percorso in mm	Tick dell'encoder sinistro	Tick dell'encoder destro	mm su tick sinistri	mm su tick destri
91	828	832	0,1099033816	0,109375
109	965	970	0,1129533679	0,112371134
98	909	882	0,1078107811	0,1111111111
105	891	941	0,1178451178	0,1115834219
98	908	951	0,1079295154	0,1151586369
104	925	938	0,1124324324	0,1108742004

Valori finali delle costanti di conversione (mm/tick):

- Costante ruota sinistra: 0,1114790994
- Costante ruota destra: 0,1117455841

Deviazioni standard (mm/tick):

- Sinistra: 0,003798618024
- Destra: 0,001941549543

Coefficienti di variazione:

- Sinistra: 0,03407471037 (3,41%)
- Destra: 0,01737473171 (1,74%)

Abbiamo ritenuto che la precisione ottenuta fosse adeguata ai nostri scopi.

Le costanti vengono usate per convertire tick in millimetri e viceversa con le seguenti formule:

- mm = tick × K
- tick = mm ÷ K

dove K è la costante specifica per ciascuna ruota.

2.2 Controllo della velocità

Ora che siamo in grado di stimare, seppur con approssimazione, le distanze percorse dalle ruote (a condizione che le nostre ipotesi siano valide), il passo successivo è progettare un algoritmo che regoli dinamicamente il duty cycle dei motori, in modo da controllare la velocità delle ruote con buona precisione.

L'implementazione specifica utilizzata nel mio progetto si trova nel file src/scheduling.c. Di seguito è riportata una descrizione dell'algoritmo in forma di pseudocodice commentato.

```
ErroreIntegrale = 0;
/*    il ciclo richiede sempre la stessa quantità di tempo per
      eseguire, rappresentata dalla variabile tempo */
while(true) {
      tick = encoder.getTicks()
      encoder.reset()

      /* calcolo quanti tick avrei dovuto fare nel ciclo precedente, basandomi
         sulla velocità e sulla durata del ciclo */
      mmObbiettivo = velocità * tempo;
      tickObbiettivo = mmObbiettivo / K;

      // calcolo l'errore dello specifico ciclo e del intero percorso
      errore = tickObbiettivo - tick;
      erroreIntegrale = erroreIntegrale + errore;

      // modifico il dutyCycle per correggere l'errore
      /* più è lungo il periodo del ciclo, più basso dovrà essere il duty cycle
         (un errore di 1 tick in un periodo di 1 secondo necessita una correzione
          della velocità molto minore rispetto a un errore di 1 tick in un periodo di 5
          millisecondi) */
      dutyCycle = errore * Kp / tempo;
      dutyCycle = dutyCycle + erroreIntegrale * Ki / tempo;
}
```

Come si può notare dallo pseudocodice, oltre alle costanti di conversione, l'algoritmo utilizza due ulteriori parametri: K_p e K_i.

- K_p regola il duty cycle in base all'errore istantaneo (in tick) tra la velocità desiderata e quella effettiva, con l'obiettivo di ridurre questo errore nel ciclo successivo.
- K_i agisce sull'errore accumulato nel tempo, correggendo eventuali scostamenti persistenti aumentando o diminuendo ulteriormente il duty cycle.

Se queste costanti sono troppo basse, l'errore tende ad accumularsi; se sono troppo alte, il sistema può diventare instabile e generare oscillazioni nella velocità. Tuttavia, la precisione nella scelta di K_p e K_i non è critica per ottenere un movimento fluido: per questo le ho determinate sperimentalmente, tramite tentativi.

Determinazione di K_p:

- Ho inizialmente impostato la durata del ciclo di controllo a 20 ms (frequenza di 50 Hz), permettendo variazioni ampie nell'errore.
- Ho fatto percorrere al robot un tratto rettilineo con una velocità obiettivo di 50 mm/s, stampando l'errore a ogni ciclo.
- Se l'errore oscillava troppo tra valori positivi e negativi, riducevo K_p.
- Se l'errore tendeva a mantenere sempre lo stesso segno, aumentavo K_p.

L'obiettivo era ottenere un errore oscillante ma con valori contenuti.

Determinazione di K_i:
Dopo aver scelto K_p, ho portato la frequenza del ciclo di controllo a 500 Hz (2 ms per ciclo) e ho regolato K_i nel seguente modo:

- Se l'errore integrale aumentava troppo durante il movimento, aumentavo K_i.
- Se il sistema mostrava oscillazioni eccessive, lo riducevo.

3 Odometria

Ora possiamo misurare le distanze percorse dalle ruote in millimetri e impostarne la velocità in millimetri al secondo. Tuttavia, non abbiamo un sistema per tracciare il percorso del CoderBot. Per risolvere questo problema, ho utilizzato il codice di odometria sviluppato da uno studente di robotica e pubblicato sul forum dell'e-learning dal professor Sorrenti, visualizzabile anche nella mia consegna nel file odometria_forum.c (cartella /extra).

3.1 Modifiche

Ho apportato diverse modifiche al codice originale, tra cui:

- Creazione di un file header per dichiarare macro, struct, variabili e funzioni condivise tra più file.
- Incapsulamento dei calcoli in una funzione find_new_pose, richiamata dal codice di scheduling.
- Correzione del calcolo dell'angolo θ: il codice originale restituiva la tangente dell'angolo invece del valore in radianti.
- Revisione del calcolo del CIR (Centro di Istante Rotazione), ora basato sulla distanza media percorsa da entrambe le ruote, anziché solo da quella destra.
- Sostituzione del vettore di posizione con uno struct, per scelta stilistica.

3.2 Funzionamento

Matrici di trasformazione omogenee

Il robot tiene traccia della propria posizione tramite due strutture:

1. Una matrice di trasformazione omogenea 2D, utile per applicare traslazioni e rotazioni attraverso moltiplicazioni tra matrici.
2. Uno struct che memorizza le coordinate x e y (in millimetri) e l'angolo θ (in radianti), usato dal controllo cartesiano e per la stampa su linea di comando della posizione.

Una matrice di trasformazione omogenea 2D ha la seguente forma:

$$T=\begin{pmatrix} \cos(\theta) & -\sin(\theta) & x \\ \sin(\theta) & \cos(\theta) & y \\ 0 & 0 & 1 \end{pmatrix}$$

Moltiplicando due matrici di trasformazione omogenee si ottiene una terza matrice di trasformazione omogenea con la seguente struttura:

$$T_3=T_1*T_2=\begin{pmatrix} \cos(\theta_1)\cos(\theta_2)-\sin(\theta_1)\sin(\theta_2) & -\sin(\theta_2)\cos(\theta_1)-\sin(\theta_1)\cos(\theta_2) & \cos(\theta_1)*x_2-\sin(\theta_1)*y_2+x_1 \\ \sin(\theta_1)\cos(\theta_2)+\sin(\theta_2)\cos(\theta_1) & -\sin(\theta_1)\sin(\theta_2)+\cos(\theta_1)\cos(\theta_2) & \sin(\theta_1)*x_2+\cos(\theta_1)*y_2+y_1 \\ 0 & 0 & 1 \end{pmatrix}$$

Si può osservare che:

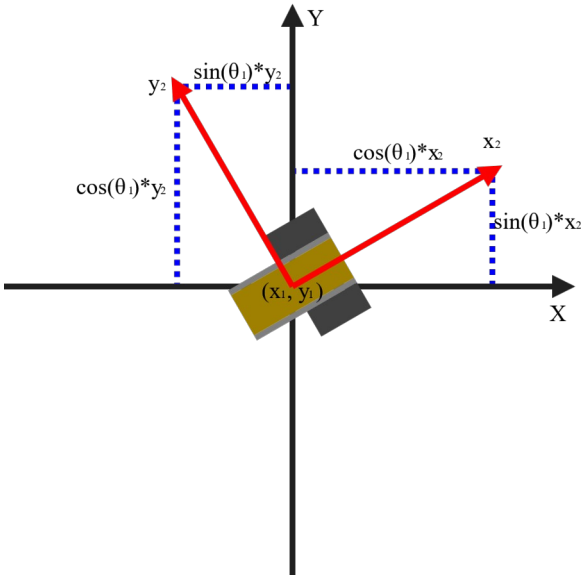
- Gli elementi nella prima riga, prima colonna e nella seconda riga, seconda colonna sono identici.
- L'elemento nella seconda riga, prima colonna è l'opposto di quello nella prima riga, seconda colonna.

Le espressioni goniometriche si possono semplificare come segue:

$$\cos(\theta_1)\cos(\theta_2)-\sin(\theta_1)\sin(\theta_2)=\cos(\theta_1+\theta_2)$$
$$\sin(\theta_1)\cos(\theta_2)+\sin(\theta_2)\cos(\theta_1)=\sin(\theta_1+\theta_2)$$

Di conseguenza, i seni e coseni presenti nella matrice risultante corrispondono a quelli dell'angolo somma $\theta_1 + \theta_2$.

L'immagine seguente mostra come le componenti x e y della matrice T_3 determinano una traslazione del robot lungo le direzioni x_2 e y_2:



Algoritmo
La funzione di odometria calcola innanzitutto la variazione dell'angolo ($\Delta\theta$) del robot seguendo questi passaggi:

- Calcola la differenza di distanza percorsa dalle due ruote:
  $\Delta mm = mm_destra - mm_sinistra$
- Usa questa differenza, insieme alla distanza tra le ruote, per determinare $\Delta\theta$ tramite la funzione atan2 della libreria matematica standard in C.

Dopo aver calcolato $\Delta\theta$, il suo valore assoluto viene confrontato con un angolo minimo definito da macro nel codice:

- Se $\Delta\theta$ è inferiore a questa soglia, si approssima il movimento a una traslazione rettilinea nella direzione x del sistema di riferimento del robot. In questo caso, si applica una semplice traslazione moltiplicando la matrice di posizione per una matrice di traslazione lungo x. La distanza della traslazione corrisponde alla media aritmetica dei millimetri percorsi dalle ruote sinistra e destra.
- Se $\Delta\theta$ è maggiore della soglia, il movimento viene trattato come un arco di cerchio e si eseguono i seguenti passaggi:
  1. Si calcola la distanza tra il centro del robot e il centro istantaneo di rotazione (CIR) lungo l'asse y del robot.
  2. Si trasla il robot verso il CIR.
  3. Si ruota la posizione del robot sommando $\Delta\theta$ all'angolo corrente.
  4. Si applica una traslazione inversa, annullando quella del punto 2.

Infine, la nuova posizione e l'angolo vengono salvati nello struct usato dal controllo cartesiano. Anche in questo caso l'angolo, calcolato tramite la funzione atan2, viene e salvato in radianti.

4 Controllo cartesiano

A questo punto, possiamo stimare, con un certo margine di errore, la posizione del robot sul piano cartesiano e controllare la velocità delle ruote. Il prossimo obiettivo è scrivere il codice per far seguire al robot una sequenza di punti predefiniti.

4.1 Modifiche

Per questa parte del progetto ho usato come base il codice dello studente Paolo Strianese, contenuto nel file controllo_cartesiano_paolo.c (cartella /extra). Ho però apportato diverse modifiche, tra cui:

- Creazione di un file header con macro, struct, variabili e funzioni condivise tra più file.
- Aggregazione delle variabili di velocità in uno struct.
- La funzione generate_arc_points scriveva i punti dell'arco in un array temporaneo per poi invertirli nell'array finale. Ho eliminato questo passaggio di ribaltamento dell'array.
- Aggiunta di una funzione per generare segmenti rettilinei tra due punti.
- Introduzione di una funzione per calcolare la distanza tra due punti, sostituendo il codice duplicato.
- La funzione incaricata di individuare il punto più vicino al robot analizzava, a ogni ciclo, l'intero insieme dei punti del percorso. Questo comportava un WCET direttamente dipendente dal numero totale di punti, che pur essendo fisso durante l'esecuzione, può variare a priori in base al percorso configurato. Di conseguenza, ogni modifica al tracciato richiederebbe un nuovo calcolo del WCET, rendendo l'approccio poco adatto a un sistema real-time. Inoltre, in percorsi con tratti sovrapposti, come una traiettoria a "otto", il robot potrebbe puntare verso un punto non corretto. L'algoritmo è stato quindi modificato per avviare la ricerca dal punto più vicino individuato nel ciclo precedente, limitandosi a un numero massimo prefissato di punti.
- Il calcolo dell'angolo ideale utilizzava in precedenza come riferimenti il punto più vicino sul percorso e il punto obiettivo. Ora, invece di approssimare la posizione del robot con il punto più vicino, l'algoritmo utilizza direttamente la posizione stimata tramite odometria, rimuovendo l'errore introdotto dall'approssimazione.
- Il robot mira a un punto più avanti nel percorso, non al successivo immediato. Questo numero (n) è ora definito tramite macro, semplificando eventuali modifiche.
- Il percorso veniva considerato completato quando l'indice del punto mirato superava quello dell'ultimo punto, causando una terminazione anticipata del programma. Ora la fine si verifica solo quando l'ultimo punto è effettivamente il più vicino.
- In caso di correzione angolare, la velocità di una ruota veniva ridotta usando la formula $\frac{1}{|\Delta\theta * 180| * \pi}$, con Δθ in radianti. Non comprendendo le motivazioni dietro questa scelta, l'ho sostituita con una funzione lineare da me definita, descritta in dettaglio più avanti.

4.2 Funzionamento

Ricerca del punto più vicino

Per calcolare la distanza tra due punti $p_1=(x_1, y_1)$ e $p_2=(x_2, y_2)$, si determina:

$$\Delta x = x_2 - x_1 \ , \Delta y = y_2 - y_1$$

Questi valori rappresentano le coordinate del punto $p_3=(\Delta x, \Delta y)$, ovvero la posizione relativa di p_2 rispetto a p_1, considerato come origine. Trattando Δx e Δy come cateti di un triangolo rettangolo, si applica il teorema di Pitagora per ottenere la distanza al quadrato:

$d^2 = \Delta x^2 + \Delta y^2$

Poiché l'elevamento al quadrato preserva l'ordinamento tra numeri positivi, confrontare le distanze al quadrato è sufficiente per identificare il punto più vicino al robot, evitando il calcolo della radice quadrata.

Calcolo del fattore di correzione della velocità
La correzione dell'angolo di orientamento del robot avviene riducendo la velocità di una ruota, moltiplicandola per un fattore compreso tra 0 e 1. Questo fattore è definito da una funzione lineare che associa l'errore angolare assoluto $\Delta\theta \in [0, \pi]$ a un valore tra c e 1, dove:

$c = f(0)$ è il valore minimo del fattore di correzione,
$f(\pi) = 1$ è il valore massimo.

La funzione è:
$f(\Delta\theta) = m * \Delta\theta + c$
con coefficiente angolare:
$m = \frac{1-c}{\pi}$
In questo modo, la correzione cresce linearmente con l'errore angolare.

Algoritmo:
L'algoritmo procede come segue:

1. Selezione del punto da raggiungere:
   Si individua il punto obiettivo, corrispondente all'n-esimo successivo del punto attualmente più vicino al robot. Il valore di n è definito tramite macro.
2. Calcolo dell'errore angolare:
   Si determina la differenza tra l'angolo di orientamento attuale del robot e l'angolo necessario per dirigersi verso il punto obiettivo.
3. Controllo del movimento:
   - Se l'errore angolare è inferiore a una soglia predefinita, il robot procede con entrambe le ruote alla velocità impostata.
   - In caso contrario, viene applicato il fattore di correzione della velocità a una delle due ruote: quella situata sul lato verso cui il robot deve ruotare viene rallentata, consentendo una correzione graduale dell'errore angolare.

5 Scheduling

Il controllo della base mobile CoderBot è stato gestito tramite scheduling real-time periodico, utilizzando l'algoritmo EDF (Earliest Deadline First). La gestione dei thread e dei mutex è stata

realizzata tramite le API di pthread.h, basandosi sullo scheletro fornito in laboratorio da Jacopo Maltagliati per lo scheduling EDF.

EDF (Earliest Deadline First)

EDF è un algoritmo di scheduling real-time ottimale per la minimizzazione del massimo ritardo di un insieme di task indipendenti e con tempi di arrivo non noti a priori. Il ritardo di una task è definito come la differenza tra il suo tempo assoluto di completamento e la sua deadline assoluta.

Il funzionamento dell'algoritmo è semplice: a ogni istante, EDF mette in esecuzione tra le task attualmente in ready queue, una delle task con deadline assoluta pari alla deadline assoluta minima attuale.

Nel caso di task periodiche, ogni task schedulata con EDF deve essere caratterizzata da tre parametri:

- WCET (Worst-Case Execution Time): indica il tempo massimo necessario per completare la task. È utilizzato dallo scheduler per calcolare il fattore di utilizzo del processore.
- Periodo: definisce l'intervallo di tempo tra due attivazioni consecutive della task.
- Deadline: specifica entro quanto tempo, a partire dall'inizio del periodo, la task deve terminare la propria esecuzione nel caso peggiore.

Implementazione in ambiente Linux

Per abilitare lo scheduling EDF nel kernel Linux, sono state utilizzate due funzioni fornite da Maltagliati durante il laboratorio:

- set_sched_deadline(uint64_t runtime_ns, uint64_t deadline_ns, uint64_t period_ns)
- dl_miss_handler()

La funzione set_sched_deadline svolge le seguenti operazioni:

1. Crea una struttura *struct sched_attr*, contenente i parametri di scheduling da passare al kernel tramite syscall. La struttura indica al kernel di usare lo scheduler EDF, con i parametri passati alla funzione, e di inviare al thread un segnale *SIGXCPU* in caso di violazione della deadline.
2. Richiede al kernel di applicare lo scheduling descritto dallo struct al thread, eseguendo una syscall. Se la syscall fallisce, viene stampato un messaggio di errore.
3. Imposta dl_miss_handler() come gestore del segnale SIGXCPU.

Quando il gestore dl_miss_handler() viene invocato (cioè quando si verifica una deadline miss), stampa un messaggio di avviso e ripristina il gestore di default per il segnale SIGXCPU. Se la deadline miss si ripete, il gestore di default terminerà il processo.

5.1 Divisione in Task

Prima di impostare i parametri di scheduling, dobbiamo dividere i compiti del software in task.

Gestione velocità e lettura tick
La gestione della velocità e la lettura dei tick di ogni ruota sono operazioni sequenziali e vanno eseguite nello stesso thread. Le due ruote sono indipendenti e possono essere gestite in task separate, quindi ho creato due task diverse, una per ogni ruota.

Odometria e controllo cartesiano
I dati dell'odometria servono solo al controllo cartesiano, e il controllo cartesiano usa solo quei dati. Se il controllo venisse eseguito più volte senza nuovi dati di odometria, sprecheremmo risorse per ricalcolare le stesse velocità. Se l'odometria venisse eseguita più volte senza un controllo in mezzo, avremmo solo calcolato dati intermedi poco utili.

Vogliamo però eseguire il controllo cartesiano il più spesso possibile, aggiornando la velocità appena arrivano nuovi dati di posizione. L'odometria limita la frequenza di aggiornamento, perché deve essere percorsa una certa distanza minima prima di aggiornare la posizione. Per questo motivo, odometria e controllo cartesiano vengono eseguiti insieme in un'unica task.

Riassumendo, il programma ha 3 task (thread):

- Controllo ruota sinistra
- Controllo ruota destra
- Aggiornamento posizione e velocità delle ruote

5.2 Impostazione di periodi e deadline

Ora che abbiamo definito le task, dobbiamo stabilire la loro frequenza di esecuzione e il WCET (Worst-Case Execution Time). Le deadline coincidono con i periodi, perché l'unico vincolo è che ogni esecuzione di una task finisca entro la fine del proprio periodo. Le due task di controllo delle ruote hanno gli stessi parametri, dato che le loro esigenze sono identiche.

Periodo delle task di controllo ruote:
L'algoritmo di controllo velocità, sviluppato seguendo le indicazioni del professore Sorrenti, richiede una frequenza di aggiornamento molto alta perché la ruota si muove principalmente per inerzia. Aggiornando il duty cycle frequentemente, con piccoli incrementi o decrementi, la ruota raggiunge gradualmente la velocità obiettivo e la mantiene stabile nel tempo. Se l'aggiornamento fosse lento, il movimento risulterebbe a scatti, perché i duty cycle impostati finirebbero per determinare direttamente la velocità della ruota invece di influenzarla gradualmente. Per questo ho impostato un periodo di 2 millisecondi (frequenza 500 Hz) per il controllo delle ruote, permettendo così un movimento fluido.

Periodo della task di odometria e controllo cartesiano:
Non stabilisco quando eseguire questa task in base al tempo trascorso, ma in base alla distanza percorsa. Il periodo deve essere abbastanza breve da farla partire subito dopo che è stata percorsa la distanza minima richiesta. Ho scelto un periodo di 10 millisecondi (frequenza 100 Hz).

Il carico sul processore è basso, quindi ho potuto impostare frequenze alte senza molti test. In futuro, se si aggiungessero operazioni più pesanti, si potrebbe ridurre la frequenza delle task attuali per risparmiare risorse.

5.3 Impostazione dei WCET

I valori di WCET (Worst Case Execution Time) consentono al kernel di prevedere se, nel peggiore dei casi, la richiesta di utilizzo del processore per eseguire il set corrente di task supererà la capacità disponibile. In tal caso, il kernel interviene, che di default significa terminare il processo. Impostare valori non troppo alti dà più flessibilità al sistema, ma valori troppo bassi possono causare il mancato rispetto delle deadline. Il tempo di esecuzione di una task è sempre non deterministico, a causa di

fattori hardware e software come cache, pipelining, interrupt e overhead del kernel, etc. Per questo, è necessario aggiungere un margine di errore.

Per calcolare il WCET ho scritto codice che, per ogni periodo, somma il tempo di utilizzo della CPU e il tempo di attesa dovuto alle sezioni critiche, aggiornando una variabile che rappresenta il tempo di esecuzione massimo del thread durante questa esecuzione del programma. Queste variabili, una per ogni thread, vengono stampate alla fine del percorso.

Entrambi i tipi di task hanno vari branch, ma con poche istruzioni, quindi possono essere considerati rumore nel tempo di esecuzione. Entrambi includono una sezione critica con accesso bloccante. Il tempo per accedere a queste sezioni è praticamente indipendente dal percorso eseguito o dalla velocità. Perciò non ho ritenuto necessario considerare scenari peggiori specifici. I test per il WCET sono stati fatti su un percorso qualunque, precisamente quello eseguito impostando la macro PATH_INDEX a 0. Ho ripetuto il percorso 10 volte e registrato i seguenti valori:

```
tempo peggiore della task di controllo del motore sinistro: 293698 ns
tempo peggiore della task di controllo del motore destro: 205468  ns
tempo peggiore della task di controllo cartesiano: 423334 ns

tempo peggiore della task di controllo del motore sinistro: 406771 ns
tempo peggiore della task di controllo del motore destro: 230155  ns
tempo peggiore della task di controllo cartesiano: 294427 ns

tempo peggiore della task di controllo del motore sinistro: 378073 ns
tempo peggiore della task di controllo del motore destro: 422083  ns
tempo peggiore della task di controllo cartesiano: 269584 ns

tempo peggiore della task di controllo del motore sinistro: 287553 ns
tempo peggiore della task di controllo del motore destro: 270937  ns
tempo peggiore della task di controllo cartesiano: 284322 ns

tempo peggiore della task di controllo del motore sinistro: 271980 ns
tempo peggiore della task di controllo del motore destro: 466146  ns
tempo peggiore della task di controllo cartesiano: 329636 ns

tempo peggiore della task di controllo del motore sinistro: 208176 ns
tempo peggiore della task di controllo del motore destro: 471719  ns
tempo peggiore della task di controllo cartesiano: 299530 ns

tempo peggiore della task di controllo del motore sinistro: 392655 ns
tempo peggiore della task di controllo del motore destro: 279271  ns
tempo peggiore della task di controllo cartesiano: 368542 ns

tempo peggiore della task di controllo del motore sinistro: 417448 ns
tempo peggiore della task di controllo del motore destro: 271821  ns
tempo peggiore della task di controllo cartesiano: 342291 ns

tempo peggiore della task di controllo del motore sinistro: 300833 ns
tempo peggiore della task di controllo del motore destro: 287448  ns
tempo peggiore della task di controllo cartesiano: 338801 ns

tempo peggiore della task di controllo del motore sinistro: 318802 ns
tempo peggiore della task di controllo del motore destro: 252500  ns
tempo peggiore della task di controllo cartesiano: 297553 ns

tempo peggiore della task di controllo del motore sinistro: 365105 ns
tempo peggiore della task di controllo del motore destro: 530259  ns
tempo peggiore della task di controllo cartesiano: 430781 ns
```

Il WCET della task di controllo dei motori è stato impostato a 1 ms. Con un tempo di esecuzione massimo misurato di 530259ns, il margine di errore è dell'88,6%. Per la task di controllo cartesiano, il WCET è stato impostato a 2 ms, a fronte di un tempo massimo di 430781ns, con un margine di errore del 364,3%
Ora che abbiamo i WCET e i periodi delle task, possiamo calcolare i rispettivi fattori di utilizzo del processore usando la formula:
$U=\frac{C_i}{T_i}$, dove C_i è il tempo di esecuzione della specifica task nel caso peggiore e T_i è il suo periodo.
Se tutte le task venissero eseguite dallo stesso core, il fattore di utilizzo totale sarebbe la somma dei singoli:
$$\frac{1\,ms}{2\,ms}+\frac{2\,ms}{10\,ms}=70\,\%$$
Questo valore è teorico e sovrastimato a causa degli ampi margini di errore nei WCET. In pratica, il carico sarà molto più basso e anche distribuito tra i vari core del Raspberry Pi.

Gestione delle sezioni critiche
L'algoritmo di scheduling EDF è ottimale solo se le task sono indipendenti, ma diventa euristico in presenza di sezioni critiche. A lezione abbiamo analizzato vari problemi legati all'accesso mutualmente esclusivo alle risorse, come l'inversione di priorità, il chained blocking e il deadlock, oltre a protocolli progettati per gestirli.

Tuttavia, dato che il progetto è semplice, i WCET hanno ampi margini di errore e le sezioni critiche sono molto brevi, non ho ritenuto necessario adottare questi algoritmi.

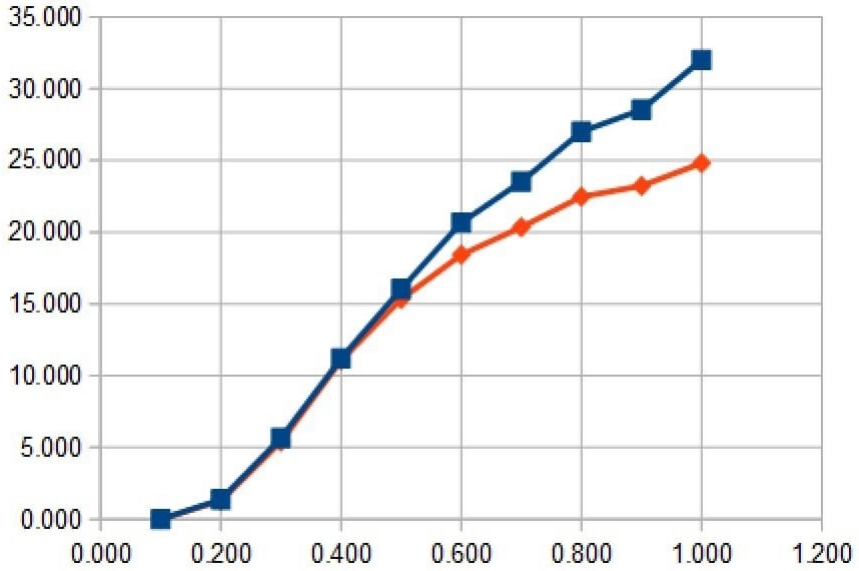
6 Fine-tuning delle costanti

Durante lo sviluppo del progetto, oltre alle costanti definite per il controllo dei motori e quelle relative allo scheduling (discusse nel capitolo precedente), sono emerse nuove costanti impostate inizialmente in modo approssimativo o "a occhio". È quindi necessario eseguire una serie di test per assegnare a queste costanti valori più adeguati.

6.1 Velocità

Duty cycle massimo
Nel codice di controllo della velocità, si assume una proporzionalità diretta tra il duty cycle e la velocità di rotazione. Questa è un'approssimazione grossolana, ma accettabile, poiché il calcolo del duty cycle non richiede particolare precisione.

All'inizio del progetto ho sviluppato un semplice programma che imposta un duty cycle su entrambi i motori e misura la media dei tick percorsi ogni 20 millisecondi. Il codice sorgente si trova nel file *trova-grafico.c*, nella cartella */extra* della consegna.
Eseguendo il programma più volte con duty cycle diversi e con il powerbank a carica massima, ho ottenuto il seguente grafico:



L’asse delle ascisse del grafico mostra il duty cycle applicato, mentre l’asse delle ordinate indica i tick medi registrati. La linea blu rappresenta la ruota sinistra, quella rossa la ruota destra.

Quando il duty cycle supera 0.6, l’incremento dei tick diventa meno marcato, soprattutto per la ruota destra è in modo più severo quando il powerbank è più scarico. Questo implica che, se si vuole sfruttare l’intero intervallo di duty cycle disponibili, si deve scegliere tra oscillazioni del robot a valori bassi e accumulo di errore a valori alti.

Questo comportamento è correlato con un notevole aumento degli episodi di undervoltage, probabilmente dovuti all’incapacità del powerbank, che alimenta anche il Raspberry Pi, di soddisfare l’elevata richiesta di corrente dei motori.

Per evitare questi problemi, ho fissato il duty cycle massimo a 0.6.

Velocità minima e massima:
Se la velocità impostata è troppo bassa, il duty cycle risulta insufficiente a muovere le ruote per diversi cicli. Solo dopo un significativo accumulo dell’errore integrale, il contributo risultante diventa abbastanza grande da generare un movimento improvviso del robot.

Al contrario, se la velocità è troppo alta, il robot raggiunge frequentemente il limite massimo del duty cycle senza riuscire a mantenere la velocità desiderata. Inoltre, velocità elevate causano un’eccessiva oscillazione dell’angolo del robot, che potrebbe essere ridotta adottando una funzione esponenziale, anziché lineare, per il calcolo del fattore di correzione angolare nel controllo cartesiano.

Non sono state ammesse velocità negative, poiché il movimento in retromarcia, pur essendo possibile, genera forti oscillazioni.

I valori minimo e massimo accettabili sono stati determinati mediante test a diverse velocità, selezionando quelli che garantivano un adeguato margine di sicurezza.

6.2 Odometria

Diverse costanti influenzano la capacità del coderbot di seguire il percorso. Per migliorarne la precisione, ho prima ottimizzato il comportamento odometrico, e solo in seguito ho considerato le discrepanze tra lo stato stimato e la posizione reale.

Strumento "visualizza percorsi.html"
L'unico modo iniziale per monitorare lo stato interno del robot era tramite stampe sulla CLI di Linux, accessibile in remoto via SSH. Tuttavia, queste stampe testuali non permettevano di valutare efficacemente il percorso sul piano cartesiano, rendendo necessario uno strumento grafico.
Per questo ho sviluppato una pagina HTML con uno script JavaScript, che offre una visualizzazione visiva del percorso, per i seguenti motivi:
Creare interfacce grafiche in HTML mi è più semplice e rapido rispetto a usare API come Java Swing.
JavaScript consente uno sviluppo veloce grazie al suo alto livello di astrazione.
Qualsiasi computer può eseguire il file senza software aggiuntivo.
La pagina si chiama "visualizza percorsi.html" ed è situata nella cartella /extra della consegna.
Permette di:

- zoomare sul percorso con la rotella del mouse;
- traslare la vista trascinando il disegno;
- visualizzare le coordinate del mouse sul disegno (in millimetri).

Nel grafico:

- la linea blu rappresenta il percorso obiettivo assegnato al robot;
- la linea nera mostra le posizioni registrate tramite odometria;
- le frecce nere indicano l'angolo theta del robot in ciascun punto.

Le array inserite nello script JavaScript provengono dalle stampe effettuate dal coderbot sulla CLI, che ho poi copiato manualmente nel file HTML. Le chiamate printf utilizzate sono ancora presenti nel codice C, attualmente commentate.

Correzione del angolo
Come detto in precedenza, il robot corregge l'angolo riducendo la velocità di una delle ruote. Questa riduzione può variare tra n% e il 100%, dove n è definito tramite macro. Dopo vari tentativi, ho scelto di impostare n a 0,5. I risultati di questi test sono visualizzabili nella pagina HTML, selezionando *"Test del fattore minimo di correzione della velocità"*.

Approssimazione al moto rettilineo
La soglia minima di variazione angolare, al di sotto della quale gli archi vengono approssimati come movimenti rettilinei, ha principalmente lo scopo di evitare divisioni per zero nei calcoli odometrici; evenienza molto rara ma possibile. Un beneficio secondario è la riduzione del costo computazionale medio della funzione. Considerate le ampie risorse computazionali disponibili sul robot, ho impostato questa costante a un valore molto basso (in radianti), corrispondente a circa 0,29 gradi di rotazione.

Distanza richiesta per aggiornare la posizione
La posizione interna del robot viene aggiornata solo quando la media delle distanze percorse dalle due ruote supera una soglia minima. Se questa soglia è zero, il coderbot approssima quasi sempre l'arco percorso come uno spostamento rettilineo. In questo caso, osservando il grafico, si nota che il robot stima di avere percorso una curva molto più ampia di quella obbiettivo, pur iniziando a ruotare su sé stesso per correggere l'angolo.
Disattivando questa approssimazione e supponendo che non avvengano divisioni per 0, il percorso che il robot pensa di avere eseguito sarà preciso indipendentemente dalla lunghezza minima del segmento, ma si riscontreranno maggiori discrepanze tra odometria e realtà a causa del numero più elevato di operazioni in virgola mobile eseguite consecutivamente e da una minore granularità relativa dei tick, portanto a stime del angolo più imprecise.
Valori alti per questa costante riducono invece la frequenza di aggiornamento della posizione, rendendo il robot meno reattivo. Nei test non sono andato oltre i 6 mm, ritenendolo un valore eccessivo.
Per verificare l'interazione tra questa costante e quella del fattore di correzione della velocità, ho svolto test variando entrambe contemporaneamente. Eventuali effetti combinati sono risultati trascurabili.
I test possono essere visualizzati nella pagina HTML selezionando *"Test della distanza minima per aggiornare la posizione"*.

Scelta del punto da mirare
Se il punto obiettivo scelto dal robot è troppo vicino alla sua posizione attuale, il robot tende a sbandare. Questo fenomeno è visibile alla fine dei percorsi nella pagina HTML, dove si nota un piccolo sbandamento dovuto alla bassa distanza dal punto mirato.
Per questo motivo, il robot non punta, se possibile, al punto immediatamente successivo a quello più vicino, ma a un punto situato ad n posizioni più avanti nel percorso. Tuttavia, non esiste una relazione fissa tra l'indice dei punti e le loro distanze reciproche o rispetto al robot.
Di conseguenza, ottimizzare questo parametro non ha senso, poiché dipende dal percorso specifico in esecuzione.
Per risolvere questo problema, sarebbe necessario modificare il codice di robotica, in modo che il punto da mirare venga scelto in base alla distanza e non all'indice.

Errore massimo delle ruote
Se una delle ruote non riesce a mantenere la velocità richiesta e accumula un errore eccessivo, il programma viene terminato. Questa scelta implementativa è motivata dai seguenti fattori:

- Il motore potrebbe trovarsi in stallo o essere parzialmente ostacolato, condizione potenzialmente dannosa per l'integrità del motore stesso.
- L'angolo di rotazione del robot dipende dal rapporto tra le rotazioni delle due ruote nel tempo. Correggere un errore accumulato in ritardo non equivale ad aver effettuato la rotazione corretta nel momento opportuno.

La definizione della soglia di errore oltre la quale interrompere l'esecuzione comporta un compromesso: se impostata troppo bassa, il robot potrebbe, soprattutto a velocità elevate, arrestarsi anche in situazioni accettabili, ad esempio all'inizio del percorso, quando le ruote devono ancora raggiungere la velocità desiderata; se impostata troppo alta, la protezione contro lo stallo risulterebbe meno efficace.
Dopo diversi test, ho deciso di impostare la soglia a 1 centimetro.

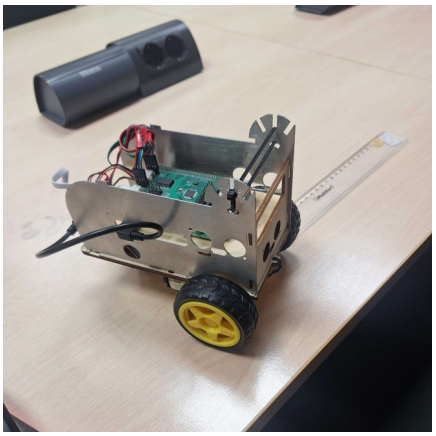
6.3 Errore reale

Anche se il coderbot ritiene di aver seguito correttamente il percorso, ciò non garantisce che la traiettoria reale corrisponda. Un accumulo di errore proporzionale alla lunghezza del percorso è inevitabile, considerando che gli unici sensori sono gli encoder. Tuttavia, possiamo cercare di ridurre questo errore.

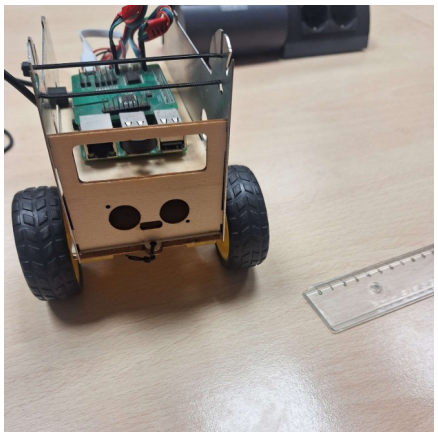
L'unica costante che ho ritenuto influente è la distanza minima da percorrere prima di aggiornare l'odometria. Per i test ho usato un percorso a semicerchio di raggio 23 cm, scelto per far spostare il robot da un'estremità all'altra di un righello da 30 cm. Ho calcolato il raggio sommando metà righello (15 cm), il margine (1 cm) e la distanza tra il centro del robot e il punto più esterno della ruota (7 cm).

Se la costante era impostata a 1 mm, il robot seguiva un arco troppo stretto e finiva sempre sopra il righello, possibilmente per un eccessiva occorrenza dell'approssimazione di archi a spostamenti rettilinei. Con valori tra 2 mm e 6 mm, il robot terminava approssimativamente nella posizione prevista. L'errore variava tra le esecuzioni, ma con la costante a 2 mm l'errore risultava solitamente minore.

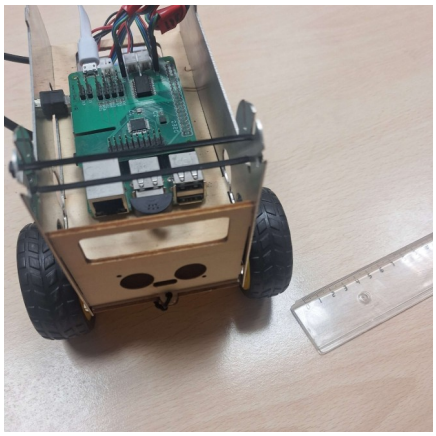
Le immagini nella pagina seguente mostrano i punti di arrivo di alcune esecuzioni.



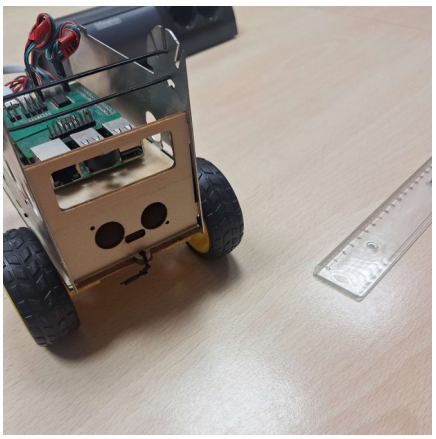
Costante a 1mm



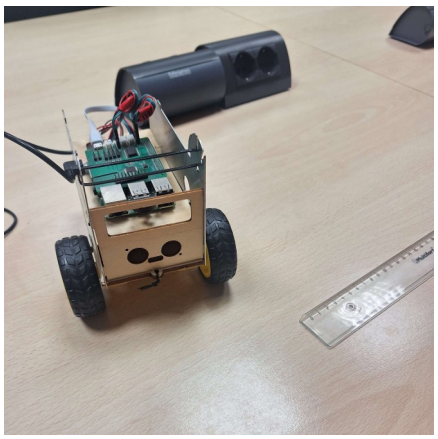
Costante a 2mm



Costante 2mm



Costante a 4mm



Costante a 6mm